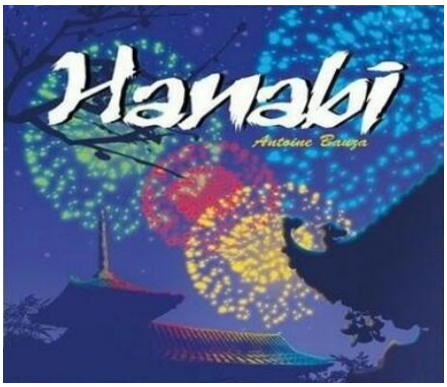
Suivi des connaissances - Projet Multi-Agents

Tom David, Pasquier Titouan, Guillais Marius, Chepy Maxim

Semestre 1 - 2025/2026





Sommaire

1 Description Générale **3**
1.1 Système Multi-Agents 3
1.2 Objectifs du projet 3
1.3 Organistation des fichiers 3

2 Agents **4**
2.1 Fonctionnement 4
2.2 Implémentation 4

3 Implémentations **5**
3.1 Hanabi 5
3.1.1 Principe 5
3.1.2 Implémentation 5
3.2 Jeu de la ville 5
3.2.1 Principe 5
3.2.2 Implémentation 5

4 Conclusion **6**
4.1 Avancé actuelle 6
4.2 Répartition des tâches 6
4.3 Améliorations 6

1 Description Générale

1.1 Système Multi-Agents

Un système multi-agents est un système composé d'un ensemble d'agents actifs dans un certain environnement et interagissant selon certaines règles.

Ce type d'implémentations a pour objectifs de décentralisé la résolution de certains problèmes. Cette méthode a été inspiré notamment par les fourmis et les abeilles, elle repose sur le fait de divisé l'objectif final en tâches plus simples réalisés par plusieurs agents autonomes.

1.2 Objectifs du projet

Pour ce projet, nous allons nous concentrer sur les systèmes multi-agents dans lesquelles les agents n'ont pas la possibilités de communiquer entre eux directement, les agents ne pourront se basé uniquement sur leurs observations de l'environnement et sur leur connaissance de la stratégie de l'autre agent.

L'objectif est de faire un modèle de ce type d'agent robuste et capable de s'adapter à n'importe quel modèle qui correspond à certaines conditions en java.

Pour ce faire nous nous sommes concentré sur 2 implémentations : Hanabi et le jeu de la ville.

1.3 Organistation des fichiers

Le dossier "core" contient tous les fichiers permettant l'implémentation de l'agent.

L'implémentation de Hanabi se trouve dans le dossier "hanabi".

Enfin, dans le dossier "ville", on retrouve les fichiers relatifs au jeu de la ville.

Pour pouvoir lancer le projet, il y a 2 fichiers "run_ville2.sh" et "run_hanabi.sh" qui éxécute respectivement le jeu de la ville et hanabi.

2 Agents

2.1 Fonctionnement

Un agent doit être capable de s'adapter à n'importe quel environnement et de décider quel action choisir à chaque pas de temps, pour ce faire il doit pouvoir se basé sur ses propres observations ainsi que sur ses actions précédentes.

De plus, il doit pouvoir être utilisé dans des implémentations très diverses sans avoir besoin de le modifier et sans qu'il ne perde en efficacité.

2.2 Implémentation

Afin de répondre au critères précédemment cités, nous avons d'abord cherché une modélisation générale qui permettrait de représenter n'importe quel environnement. Après réflexion rapide, nous nous sommes rendu compte que c'était impossible et en avons déduit qu'il fallait que la modélisation soit fourni au cas par cas.

Nous avons d'ailleurs eut la même réflexion pour la stratégie utilisée, ce que nous allons également appellé politique.

Pour ce qui est du code, nous avons créer une classe Agent qui prend un historique et une politique en argument, une politique étant une interface dont la réalisation concrète doit être fournie par l'implémentation. Pour l'historique, c'est une Map d'actions et d'observation dont ces derniers seront récupérés dans l'implémentation.

Nous avons également créer une politique par défaut, qui s'appelle politique random et qui fait une aléatoire à chaque pas de temps.

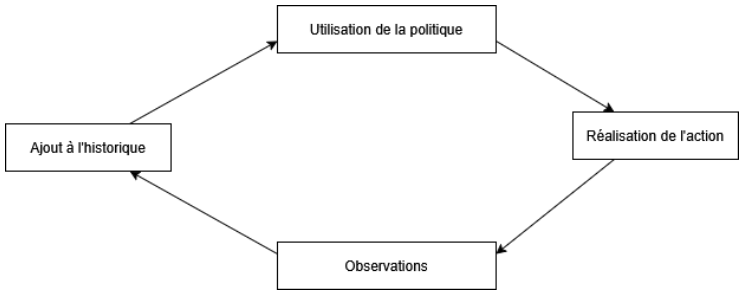


FIGURE 1 – Cycle d'utilisation de l'agent

3 Implémentations

3.1 Hanabi

3.1.1 Principe

Hanabi est un jeu de société coopératif dont l'objectif est pour chacune des 5 couleurs de placer les dans l'ordre de 1 à 5.

La particularité est que chaque joueur ne voit pas ses propres cartes mais au contraire voit celles des autres joueurs, à chaque tour un joueur à la possibilité de jouer une de ses cartes ou de donner un indice à un autre joueur permettant d'indiquer de quoi est composé la main de ce dernier.

3.1.2 Implémentation

Nous avons commencé par créer une classe Carte, chaque carte prenant un numéro et une couleur. Puis nous avons utilisé une Map<Agent, List<Carte» afin de représenter les mains des joueurs, la pioche est représenté par une List<Carte>

3.2 Jeu de la ville

3.2.1 Principe

Le jeu de la ville est une simulation dans laquelle plusieurs individus doivent se retrouver dans une ville sans pouvoir communiquer entre eux.

Les agents possèdent plusieurs choix à chaque tour, aller à une case voisines, rester sur place, laisser un message sur la case actuelle.

De plus durant la simulation, la ville évolue, en effet certaines routes peuvent se retrouver bloqué ou à minima ralenti.

3.2.2 Implémentation

Pour l'implémentation du jeu de la ville, nous avons décidé de le modélisé par un graphe, chaque sommet représente un lieu-dit (Métro, Centre-ville ...) et chaque arête représente un chemin permettant d'aller d'un lieu à l'autre.

Pour l'instant, nous devons construire la ville manuellement ainsi que décidé quelles routes sont bloquées mais nous avons pour objectif d'automatisé ces étapes ainsi que de rendre dynamique le blocage des routes, c'est-à-dire qu'une route puisse se bloquer en cours de simulation.

Nous allons également pondéré les arêtes du graphe afin de représenté le temps que prend le trajet d'un lieu à un autre. De plus, il nous suffira d'augmenter le poids de l'arête à une valeur très grande pour représenté le fait qu'elle est bloqué.

4 Conclusion

4.1 Avancé actuelle

Nous avons donc actuellement implémenté un système d'agent fonctionnelle ainsi que 2 implémentations distinctes (Jeu de la vie, Hanabi).

Nous n'avons cependant pas encore de politique adapté à chacune des implémentations.

4.2 Répartition des tâches

Tom David : Implémentation du jeu de la ville, Implémentation de l'agent

Titouan Pasquier : Implémentation d'Hanabi

Marius Guillais : Implémentation du système de message, Implémentation de l'historique, implémentation du jeu de la ville

Maxim Chepy : Travail sur les politiques, Ecriture du rapport

4.3 Améliorations

Nous allons désormais nous intéressé à la mise en place de politiques complexes pour chacune des implémentations afin qu'ils puissent être utilisé par nos agents.

Nous pourrons par exemple réfléchir à des politiques composé d'un leader et de suiveurs, c'est-à-dire un agent qui fait les choix principaux et les autres qui s'adapte à ce dernier, mais également des politiques plus naïves ne prenant que très peu les choix des autres agents en compte, ou d'autre qui permettront de changer l'objectif en cours de simulation.